
OPERATIONS • RENDER + NEON

Deployment Guide

How the live portfolio is built, deployed, updated and repaired

WEBSITE

portfolio-web-asu3.onrender.com

ADMIN

portfolio-api-9028.onrender.com/admin/

HOSTING

Render free tier, Singapore

API

portfolio-api-9028.onrender.com/api

DATABASE

PostgreSQL on Neon (Singapore)

REVISED

25 September 2026

The one rule to remember: text lives in the database and is edited in the admin. Files (photo, CV, screenshots) live in the *repository* and reach the site only through `git push`. Anything uploaded through the admin disappears.

1. How the system fits together
2. What happens when you push
3. Environment variables
4. Where each kind of content lives
5. Workflows — making common changes
6. Running a command against production
7. The admin account
8. Checklists — before and after every deploy
9. Free-tier behaviour
10. Troubleshooting
11. Rules — what never to do
12. Custom domain (optional)
13. Open item at time of writing

01 How the system fits together

Two Render services and one Neon database. Both services build from the same GitHub repository, `main` branch.

Visitor → portfolio-web (static React build) → portfolio-api (Django + gunicorn) → Neon PostgreSQL

portfolio-api → WhiteNoise → backend/media/ (photo, CV, screenshots – committed to git)

PIECE	RENDER TYPE	ROOT DIR	WHAT IT DOES
portfolio-web	Static Site	frontend	The React single-page app. Never sleeps; loads instantly.
portfolio-api	Web Service (Python 3.12)	backend	The REST API, the admin CMS, and the media files. Sleeps after 15 minutes idle.
Neon	External	—	PostgreSQL. Holds every piece of text, every project, every contact message. Survives deploys.

WHY NEON, NOT RENDER POSTGRES

Render's free database is deleted after 30 days. Neon's free tier does not expire.

Why media is committed to git

Render's disk is **ephemeral**: it is rebuilt from the repository on every deploy, and anything written to it at runtime is lost. In production Django does not serve media itself; WhiteNoise does, and WhiteNoise only knows about files that existed when the server started. So a file is served only if it is in `backend/media/` in the repository. This is why `backend/media/` is deliberately **not** in `.gitignore`.

02 What happens when you push

A push to `main` triggers both services to rebuild automatically. Changing an environment variable in the Render dashboard also triggers a redeploy of that service.

2.1 Backend — `backend/build.sh`

Every step is safe to repeat. Only step 6 can change content you edited in the admin (see 5.2).

#	STEP	WHAT IT DOES
1	<code>pip install -r requirements/prod.txt</code>	Installs Django, DRF, gunicorn, WhiteNoise.
2	<code>collectstatic</code>	Gathers admin CSS/JS for WhiteNoise.
3	<code>migrate</code>	Applies any new database migrations to Neon.
4	<code>seed_portfolio</code>	Only if the database has no projects. Loads all content on a brand-new database.
5	<code>attach_media</code>	Runs when a project has no cover, or the profile's photo or CV is empty <i>or points at a file that no longer exists</i> . Links records to the committed files.
6	<code>sync_profile_copy</code>	Pushes profile <i>wording</i> from <code>seed_portfolio.py</code> to the live database. Never touches photo, CV or links.
7	<code>set_video_urls</code>	Sets YouTube walkthrough links on projects that have none. Links edited in the admin are left alone.
8	<code>create admin</code>	Creates the admin user from <code>ADMIN_USERNAME</code> / <code>ADMIN_PASSWORD</code> if that user does not exist yet.

Then Render starts the server:

```
gunicorn config.wsgi:application --bind 0.0.0.0:$PORT
```

Render's health check calls `/api/profile/`. If it fails, the old version keeps running.

2.2 Frontend

```
npm ci && npm run build # publishes frontend/dist
```

- `VITE_API_URL` is **baked in at build time**. Changing it requires a redeploy of *portfolio-web*, not just the backend.
- A **rewrite** rule `/* → /index.html` makes URLs like `/projects/micromart` work on refresh. It must be *Rewrite*, not *Redirect*. Render ignores the `_redirects` file in `public/`.

2.3 Timeline of a normal deploy

1. `git push` — both services start building.
2. Frontend finishes in about 1–2 minutes.
3. Backend takes about 3–5 minutes (dependency install dominates).
4. Watch progress in Render → service → **Events / Logs**.
5. Run the after-deploy checklist (section 08).

03 Environment variables

Set in Render → service → **Environment**. Never commit them. Locally the same names live in `backend/.env`, which is git-ignored.

3.1 portfolio-api

VARIABLE	VALUE	NOTES
PYTHON_VERSION	3.12.0	
SECRET_KEY	long random string	Never reuse the development key.
DEBUG	False	Must be False in production.
USE_SQLITE	False	True would ignore Neon.
DATABASE_URL	Neon connection string	Contains the database password — treat as secret.
ALLOWED_HOSTS	portfolio-api-9028.onrender.com	Wrong value → <code>DisallowedHost</code> / 400.
CORS_ALLOWED_ORIGINS	https://portfolio-web-asu3.onrender.com	Wrong value → browser blocks every API call.
CSRF_TRUSTED_ORIGINS	https://portfolio-web-asu3.onrender.com	Comma-separate several origins.
SECURE_SSL_REDIRECT	False	Render already handles HTTPS; True causes a redirect loop.
ADMIN_USERNAME	your choice	Used only to create the first admin.
ADMIN_PASSWORD	strong password	Remove once the account exists (section 07).
ADMIN_EMAIL	optional	

Optional — email notification for contact messages

Messages are always stored in the database and visible in the admin. To also be emailed, set `EMAIL_BACKEND` (`django.core.mail.backends.smtp.EmailBackend`), `EMAIL_HOST`, `EMAIL_PORT`, `EMAIL_USE_TLS`, `EMAIL_HOST_USER`, `EMAIL_HOST_PASSWORD` and `CONTACT_NOTIFY_EMAIL`. For Gmail, use an *app password*, not your account password.

3.2 portfolio-web

VARIABLE	VALUE	NOTES
VITE_API_URL	https://portfolio-api-9028.onrender.com/api	The <code>/api</code> suffix is required. Without it every request 404s.

IF A HOSTNAME EVER CHANGES

Update `ALLOWED_HOSTS`, `CORS_ALLOWED_ORIGINS`, `CSRF_TRUSTED_ORIGINS` and `VITE_API_URL` together, then redeploy **both** services.

04 Where each kind of content lives

Almost every deployment problem comes from mixing these up.

CONTENT	STORED IN	HOW TO CHANGE IT	NEEDS A PUSH?
Name, tagline, about text, links, availability	Neon database	Admin → Profiles → Profile	No — live instantly
Project text, tech stack, video link, order	Neon database	Admin → Projects	No
Skills, education	Neon database	Admin	No
Contact messages	Neon database	Admin → Contact (inbox)	—
Profile photo, CV	Repository: <code>backend/media/</code>	<code>update_profile_media</code> (5.3)	Yes
Project screenshots	Repository: <code>backend/media/</code>	See 5.4	Yes
Page design, layout, code	Repository	Edit code	Yes
Sitemap, robots.txt	Repository: <code>frontend/public/</code>	<code>generate_sitemap</code>	Yes

WHY ADMIN UPLOADS FAIL

The admin's upload button saves the file to Render's disk and points the database at it. WhiteNoise never serves the file because it did not exist at startup, so the site shows a broken image. The next deploy deletes the file anyway. **Use the admin for text only.**

05 Workflows — making common changes

Backend commands run from `backend/` with the virtual environment active (`venv\Scripts\activate` on Windows). Frontend commands run from `frontend/` .

5.1 Change code (any feature, fix or design change)

```
# 1. Test locally
cd backend && python manage.py test          # all backend tests must pass
cd frontend && npm test && npm run lint && npm run build

# 2. Look at what you are about to commit
git status
git diff

# 3. Commit and push
git add <files>
git commit -m "Describe the change"
git push
```

Then watch both services in Render and run the checklist in section 08.

5.2 Change text on the site

1. Open <https://portfolio-api-9028.onrender.com/admin/> and sign in.
2. Edit the Profile, a Project, a Skill or an Education entry. Save.
3. Reload the website. No deploy needed.

Admin edits are never overwritten by a deploy, with one exception: the profile's narrative fields (tagline, hero intro, about text and similar) are re-synced from `seed_portfolio.py` on every deploy by `sync_profile_copy` . **If you edit profile wording in the admin, make the same edit in `seed_portfolio.py`** , or the next deploy puts the old wording back.

5.3 Replace the profile photo or CV

```
# 1. Drop the new file into the git-ignored source folder
#   photo -> images/      CV (PDF) -> cv/

# 2. Optimise it into backend/media/ and update the local database
python manage.py update_profile_media --dry-run      # preview
python manage.py update_profile_media               # or --photo-only / --cv-only

# 3. Commit the media and push
git add backend/media
git commit -m "Update profile photo"
git push
```

On deploy, `build.sh` notices the live database points at a file that no longer exists and re-links it to the newest file in `media/profile/` . No admin step is needed.

If you replace the photo but keep the *same filename*, the live record is still valid, so nothing needs re-linking; browsers may show the old picture until their cache expires.

5.4 Change project screenshots

Screenshot filenames are listed by hand in two commands. Both need updating:

1. Put the new screenshots in `images/<project-folder>/`.
2. In `backend/apps/projects/management/commands/import_media.py`, update `COVERS` and `CAPTIONS` for that project if the cover or captions change.
3. Run `python manage.py import_media`. It optimises the images into `backend/media/projects/` and updates your *local* database.
4. In `backend/apps/projects/management/commands/attach_media.py`, update `COVERS` and `GALLERIES` to the new output filenames.
5. Commit `backend/media` and both command files, then push.
6. The live database still points at the old gallery. Re-link it with `attach_media --force` against production (section 06).

NOTE

`attach_media --force` rebuilds every gallery, so captions return to the values in `attach_media.py`. Keep captions edited in the admin in step with that file.

5.5 Add a new project

1. Create the project and its text in the admin (or add it to `seed_portfolio.py`).
2. Add its screenshots as in 5.4, adding the new slug to the mappings in both commands.
3. Regenerate the sitemap, then commit and push:

```
python manage.py generate_sitemap --base-url https://portfolio-web-asu3.onrender.com
```

5.6 Change a dependency or the database schema

- **New Python package:** add it to `backend/requirements/base.txt` (or `prod.txt` if production-only). Render installs it on the next build.
- **Model change:** run `python manage.py makemigrations` locally, commit the migration file. `build.sh` applies it to Neon. Never edit a migration that is already deployed.
- **New npm package:** `npm install <pkg>` and commit both `package.json` and `package-lock.json`. `npm ci` fails if they disagree.
- **New environment variable:** add it to `backend/.env.example` with a comment, and to the Render dashboard *before* pushing code that needs it.

06 Running a command against production

Render's free tier has no shell. To run a management command on the live database, run it on your own machine with `DATABASE_URL` pointed at Neon. Your local `backend/media/` must match what is pushed, because file checks read your local disk.

Git Bash

```
cd backend
DATABASE_URL="<Neon connection string>" USE_SQLITE=False python manage.py <command>
```

PowerShell

```
cd backend
$env:DATABASE_URL="<Neon connection string>"; $env:USE_SQLITE="False"
python manage.py <command>
Remove-Item Env:DATABASE_URL, Env:USE_SQLITE      # back to local afterwards
```

Copy the connection string from Render → portfolio-api → Environment, or from the Neon dashboard.

COMMAND	USE IT TO
<code>changepassword <username></code>	Reset the admin password.
<code>attach_media</code>	Repair a broken photo or CV link immediately, without waiting for a deploy.
<code>attach_media --force</code>	Re-link every cover, gallery, photo and CV to the committed files.
<code>sync_profile_copy</code>	Push profile wording from <code>seed_portfolio.py</code> .

BE CAREFUL

This is the real database. Never run `seed_portfolio`, `flush`, `import_media` or `update_profile_media` this way. Unset the variables afterwards so later local commands do not touch production by accident.

07 The admin account

- **URL:** `https://portfolio-api-9028.onrender.com/admin/`
- **Credentials:** the values of `ADMIN_USERNAME` and `ADMIN_PASSWORD` in Render → portfolio-api → Environment. They are not stored anywhere in the repository.
- **Once you can log in**, delete `ADMIN_PASSWORD` from Render. The account stays; the build only creates it when it does not exist and never changes an existing password.
- **Forgotten password:** `changepassword` against production (section 06).
- Change the password from inside the admin (top right → Change password) whenever you like.

08 Checklists

8.1 Before every push

- ☐ `python manage.py test` passes
- ☐ `npm test`, `npm run lint` and `npm run build` pass
- ☐ `git status` shows only files you meant to change
- ☐ No `.env` file tracked: `git ls-files | grep -c "\.env$" prints 0`
- ☐ New images or CV are in `backend/media/` and staged
- ☐ New environment variables already added in Render
- ☐ New migrations committed

8.2 After every deploy

The first request may take about 50 seconds if the backend was asleep.

- ☐ Both services show **Live** in Render, with the latest commit
- ☐ `/api/profile/` returns JSON
- ☐ Hero shows your name and **photo**
- ☐ All project cards show screenshots
- ☐ A case study opens with its diagram and gallery
- ☐ Refreshing `/projects/micromart` does not 404
- ☐ Résumé downloads
- ☐ A test contact message appears in the admin
- ☐ Light/dark toggle works

Quick checks from a terminal (Git Bash)

```
# Which photo does the live API point at?
curl -s https://portfolio-api-9028.onrender.com/api/profile/ | grep -o '"photo": "[^"]*"'

# Does that file load? (expect 200)
curl -s -o /dev/null -w "%{http_code}\n" <photo URL from above>

# Does client-side routing survive a refresh? (expect 200)
curl -s -o /dev/null -w "%{http_code}\n" https://portfolio-web-asu3.onrender.com/projects/micromart
```

09 Free-tier behaviour

BEHAVIOUR	EFFECT
Backend sleeps after 15 minutes idle	The next visitor waits about 50 seconds for project data.
Static frontend never sleeps	The page shell appears instantly.
Disk is ephemeral	Runtime uploads are lost on every deploy (section 01).
No shell	Run commands from your machine (section 06).

Before sending the link to a recruiter, open the site yourself a minute earlier to wake the backend. Render's paid always-on instance removes the cold start; everything else can stay free.

10 Troubleshooting

SYMPTOM	CAUSE	FIX
Profile photo broken	Database points at a file that is not committed (uploaded via admin, or replaced and deleted)	Commit the photo and push; or <code>attach_media</code> against production
All images 404	<code>backend/media/</code> not committed	Commit and push it
Page loads but no project data	Backend asleep	Wait about 50 s and reload
Browser console: CORS error	<code>CORS_ALLOWED_ORIGINS</code> lacks the frontend URL	Fix the variable (no trailing slash)
Every API call 404s	<code>VITE_API_URL</code> missing <code>/api</code>	Fix it, redeploy <i>portfolio-web</i>
400 Bad Request / <code>DisallowedHost</code>	<code>ALLOWED_HOSTS</code> wrong	Set it to the API hostname
Too many redirects	<code>SECURE_SSL_REDIRECT=True</code>	Set it to <code>False</code>
Refresh on a case study 404s	Rewrite rule missing	Add <code>/* → /index.html</code> , action Rewrite
Admin CSRF failure on login	<code>CSRF_TRUSTED_ORIGINS</code> wrong	Include the relevant https origin
Build: bad <code>interpreter</code>	<code>build.sh</code> saved with Windows line endings	<code>.gitattributes</code> forces LF; re-add the file
Build: <code>npm ci</code> fails	<code>package-lock.json</code> out of date	Run <code>npm install</code> , commit the lock file
Build: module not found	Package missing from requirements	Add it to <code>requirements/</code>
Profile wording reverted after deploy	<code>sync_profile_copy</code> restored the seed text	Edit <code>seed_portfolio.py</code> too (5.2)
Contact form: 429	Rate limit, 5 per hour per IP	Intended. Wait an hour

For any other failure, read Render → service → **Logs**. The failing command and its error are near the end of the build log.

11 Rules — what never to do

- **Never upload images or the CV through the admin.** Commit them (5.3, 5.4).
- **Never add `backend/media/` to `.gitignore`.** The site's images would vanish on the next deploy.
- **Never commit `.env` or paste `SECRET_KEY`, `DATABASE_URL` or passwords anywhere public.**
- **Never commit `images/` or `cv/`.** They hold full-resolution originals and personal details; only the optimised copies belong in git.
- **Never set `DEBUG=True` in production.** It exposes settings and stack traces to visitors.
- **Never run `seed_portfolio` against production.**
- **Keep the API read-only except `POST /api/contact/`.**
- **Push only after both test suites pass.**

12 Custom domain (optional)

1. Buy a domain, then Render → portfolio-web → Settings → **Custom Domain**. Follow the DNS instructions.
2. Add the new origin to `CORS_ALLOWED_ORIGINS` and `CSRF_TRUSTED_ORIGINS` on the backend.
3. Update the canonical link and both `og:url` tags in `frontend/index.html`.
4. Regenerate the sitemap with the new `--base-url`, commit, and push.

13 Open item at time of writing

PROFILE PHOTO – FIX WRITTEN, NOT YET DEPLOYED

On 25 September 2026 the live API pointed the profile photo at `media/profile/raihan2.jpg`, an admin upload that returns 404. `attach_media.py` and `build.sh` have been changed so a deploy repairs references to missing files, but these changes are still uncommitted. Commit and push them (with a new photo from 5.3 if wanted) to restore the photo.

Source: `docs/DEPLOYMENT_GUIDE.html`. Regenerate the PDF with `docs/render-prd.mjs` (see `docs/README.md`). Update this guide whenever hosting, environment variables or `build.sh` change.